hw3

Yifeng Huang

This is the link to the repo: https://github.com/YifengHuang-Bob/Homework3

Problem 1

(a) Download and merge the datasets

```
library(haven)

aux <- read_xpt("data/AUX_I.xpt")
demo <- read_xpt("data/DEMO_I.xpt")

nhanes <- merge(aux, demo, by = "SEQN")

dim(nhanes)
```

```
[1] 4582  119
```

(b) Data cleaning

```
nhanes$RIAGENDR <- factor(
  nhanes$RIAGENDR,
  levels = c(1, 2),
  labels = c("Male", "Female")
)
```

```
nhanes$DMDCITZN[nhanes$DMDCITZN %in% c(7, 9)] <- NA

nhanes$DMDCITZN <- factor(
  nhanes$DMDCITZN,
  levels = c(1, 2),
  labels = c("Citizen", "Not a citizen")
)
```

```r
nhanes$INDHHIN2[nhanes$INDHHIN2 %in% c(77, 99)] <- NA

nhanes$INDHHIN2 <- factor(
  nhanes$INDHHIN2,
  levels = c(1:10, 14, 15, 13, 12),
  labels = c(
    "$0-$4,999",
    "$5,000-$9,999",
    "$10,000-$14,999",
    "$15,000-$19,999",
    "$20,000-$24,999",
    "$25,000-$34,999",
    "$35,000-$44,999",
    "$45,000-$54,999",
    "$55,000-$64,999",
    "$65,000-$74,999",
    "$75,000-$99,999",
    "$100,000 and over",
    "Under $20,000 (broad)",
    "$20,000 and over (broad)"
  )
)
```

The original income codes do not follow a strictly increasing income order. I rearranged the detailed income categories from lowest to highest and placed the two broad, overlapping categories separately at the end, as they cannot be precisely ordered among the detailed income categories.

(c) Poisson regression models

```r
nhanes$income_numeric <- as.numeric(nhanes$INDHHIN2)

nhanes$income_numeric[nhanes$income_numeric > 12] <- NA
```

```r
model_1r <- glm(
  AUXTWIDR ~ RIAGENDR,
  data = nhanes,
  family = poisson(link = "log")
)
```

```r
model_2r <- glm(
  AUXTWIDR ~ RIAGENDR + DMDCITZN + DMDHHSZA + income_numeric,
```

```
  data = nhanes,
  family = poisson(link = "log")
)
```

```
model_1l <- glm(
  AUXTWIDL ~ RIAGENDR,
  data = nhanes,
  family = poisson(link = "log")
)
```

```
model_2l <- glm(
  AUXTWIDL ~ RIAGENDR + DMDCITZN + DMDHHSZA + income_numeric,
  data = nhanes,
  family = poisson(link = "log")
)
```

```
irr_table <- data.frame(
  Predictor = c(
    "Intercept (exp(coefficient))",
    "Female",
    "Not a citizen",
    "Children (per unit)",
    "Income level (per unit)"
  ),
  `1R` = c(exp(coef(model_1r)), rep(NA_real_, 3)),
  `2R` = exp(coef(model_2r)),
  `1L` = c(exp(coef(model_1l)), rep(NA_real_, 3)),
  `2L` = exp(coef(model_2l)),
  check.names = FALSE
)

options(knitr.kable.NA = "-")
```

```
knitr::kable(
  irr_table,
  row.names = FALSE,
  digits = 4,
  caption = "Exponentiated coefficients from the four Poisson regression models"
)
```

Predictor	1R	2R	1L	2L
Intercept (exp(coefficient))	84.3174	88.4374	84.7584	88.8213
Female	1.0093	1.0150	1.0130	1.0187
Not a citizen	-	1.0469	-	1.0218
Children (per unit)	-	0.9944	-	0.9830
Income level (per unit)	-	0.9921	-	0.9935

```
model_stats <- data.frame(
  Statistic = c("Sample size", "Pseudo-R2 (deviance-based)", "AIC"),
  `1R` = c(
    format(nobs(model_1r), big.mark = ","),
    sprintf("%.6f", 1 - deviance(model_1r) / model_1r$null.deviance),
    format(round(AIC(model_1r)), big.mark = ",")
  ),
  `2R` = c(
    format(nobs(model_2r), big.mark = ","),
    sprintf("%.6f", 1 - deviance(model_2r) / model_2r$null.deviance),
    format(round(AIC(model_2r)), big.mark = ",")
  ),
  `1L` = c(
    format(nobs(model_1l), big.mark = ","),
    sprintf("%.6f", 1 - deviance(model_1l) / model_1l$null.deviance),
    format(round(AIC(model_1l)), big.mark = ",")
  ),
  `2L` = c(
    format(nobs(model_2l), big.mark = ","),
    sprintf("%.6f", 1 - deviance(model_2l) / model_2l$null.deviance),
    format(round(AIC(model_2l)), big.mark = ",")
  ),
  check.names = FALSE
)

knitr::kable(
  model_stats,
  row.names = FALSE,
  caption = "Model statistics for the four Poisson regression models"
)
```

Statistic	1R	2R	1L	2L
Sample size	4,149	3,705	4,103	3,665
Pseudo-R2 (deviance-based)	0.000106	0.005717	0.000200	0.003631
AIC	96,618	84,916	98,685	86,598

(d) Hypothesis tests

```
gender_test <- coef(summary(model_2l))["RIAGENDRFemale", ]

gender_test
```

```
    Estimate   Std. Error      z value     Pr(>|z|)
1.857273e-02 3.599573e-03 5.159705e+00 2.473393e-07
```

```
exp(coef(model_2l)["RIAGENDRFemale"])
```

```
RIAGENDRFemale
      1.018746
```

```
emm_gender <- emmeans::emmeans(
  model_2l,
  ~ RIAGENDR
)

summary(emm_gender, type = "response")
```

```
 RIAGENDR rate    SE  df asymp.LCL asymp.UCL
 Male     84.8 0.252 Inf      84.3      85.3
 Female   86.3 0.249 Inf      85.9      86.8

Results are averaged over the levels of: DMDCITZN
Confidence level used: 0.95
Intervals are back-transformed from the log scale
```

```
gender_diff <- pairs(
  emmeans::regrid(emm_gender, transform = "response"),
  reverse = TRUE
```

```
)

summary(gender_diff, infer = c(TRUE, TRUE))
```

```
 contrast      estimate    SE  df asymp.LCL asymp.UCL z.ratio p.value
 Female - Male     1.59 0.308 Inf     0.986      2.19   5.161 <0.0001

Results are averaged over the levels of: DMDCITZN
Confidence level used: 0.95
```

Test 1 — IRR Comparison

For Model 2L, the estimated female-to-male IRR is 1.0187. The Wald test gives $z = 5.160$ and $p = 2.47 \times 10^{-7}$. Therefore, we reject the null hypothesis that the IRR equals 1 at the 5% significance level. Holding other covariates constant, females have approximately 1.87% higher expected left-ear tympanometric width than males.

Test 2 — Predicted Value Comparison

The adjusted predicted values are 84.8 for males and 86.3 for females. The estimated difference (Female – Male) is 1.59, with a 95% confidence interval of $[0.986, 2.19]$. The test gives $z = 5.161$ and $p < 0.0001$. Since the confidence interval excludes 0, we reject the null hypothesis of equal predicted values at the 5% significance level.

Note on Overdispersion

These results are based on the standard Poisson model. Since the model shows evidence of overdispersion, the reported p-values and confidence intervals should be interpreted cautiously.

Problem 2 - Sakila

```
library(DBI)

con <- dbConnect(
  RSQLite::SQLite(),
  "data/sakila_master.db"
)
```

(a) Customers and active percentages by store

Method 1 - SQL + R

```
customer_data <- dbGetQuery(
  con,
  "SELECT store_id, active FROM customer"
)

customer_counts <- table(customer_data$store_id)

active_percent <- tapply(
  customer_data$active == "1",
  customer_data$store_id,
  mean
) * 100

result_r <- data.frame(
  store_id = as.integer(names(customer_counts)),
  customers = as.integer(customer_counts),
  percent_active = as.numeric(active_percent)
)

knitr::kable(
  result_r,
  digits = 2,
  row.names = FALSE,
  caption = "Customer counts and active percentages by store (SQL + R)"
)
```

store_id	customers	percent_active
1	326	97.55
2	273	97.44

Method 2 - Single SQL Query

```
result_sql <- dbGetQuery(
  con,
  "SELECT
     store_id,
     COUNT(*) AS customers,
     ROUND(
       100.0 * SUM(CASE WHEN active = '1' THEN 1 ELSE 0 END) / COUNT(*),
```

```
       2
     ) AS percent_active
   FROM customer
   GROUP BY store_id
   ORDER BY store_id"
)

knitr::kable(
  result_sql,
  digits = 2,
  row.names = FALSE,
  caption = "Customer counts and active percentages by store (SQL only)"
)
```

store_id	customers	percent_active
1	326	97.55
2	273	97.44

Performance comparison

```
bench_a <- microbenchmark::microbenchmark(
  SQL_R = {
    data_r <- dbGetQuery(
      con,
      "SELECT store_id, active FROM customer"
    )

    counts_r <- table(data_r$store_id)

    percent_r <- tapply(
      data_r$active == "1",
      data_r$store_id,
      mean
    ) * 100

    data.frame(
      store_id = as.integer(names(counts_r)),
      customers = as.integer(counts_r),
      percent_active = as.numeric(percent_r)
    )
  },
  SQL_only = {
```

```
    dbGetQuery(
      con,
      "SELECT
         store_id,
         COUNT(*) AS customers,
         ROUND(
           100.0 * SUM(CASE WHEN active = '1' THEN 1 ELSE 0 END) / COUNT(*),
           2
         ) AS percent_active
       FROM customer
       GROUP BY store_id
       ORDER BY store_id"
    )
  },
  times = 100L
)

bench_summary <- summary(bench_a, unit = "us")

median_sql_r <- bench_summary$median[bench_summary$expr == "SQL_R"]
median_sql_only <- bench_summary$median[bench_summary$expr == "SQL_only"]

bench_summary
```

```
      expr   min     lq    mean median     uq    max neval
1    SQL_R 488.9 537.95 606.947 572.85 616.75 1261.1   100
2 SQL_only 206.1 238.95 281.264 260.20 284.45  696.0   100
```

The microbenchmark results show that the SQL-only approach is faster than the SQL + R approach. Over 100 evaluations, the median execution times were approximately 260.20 and 572.85 microseconds, respectively. Thus, the SQL-only approach was about 2.20 times faster in this benchmark, likely because the aggregation was performed directly in the database.

(b) Staff names and countries

Method 1 - SQL + R

```
staff_data <- dbGetQuery(
  con,
  "SELECT staff_id, first_name, last_name, address_id FROM staff"
)
```

```r
address_data <- dbGetQuery(
  con,
  "SELECT address_id, city_id FROM address"
)

city_data <- dbGetQuery(
  con,
  "SELECT city_id, country_id FROM city"
)

country_data <- dbGetQuery(
  con,
  "SELECT country_id, country FROM country"
)

staff_info <- merge(staff_data, address_data, by = "address_id")
staff_info <- merge(staff_info, city_data, by = "city_id")
staff_info <- merge(staff_info, country_data, by = "country_id")

result_r_b <- data.frame(
  staff_id = staff_info$staff_id,
  staff_name = paste(staff_info$first_name, staff_info$last_name),
  country = staff_info$country
)

result_r_b <- result_r_b[order(result_r_b$staff_id), ]

knitr::kable(
  result_r_b,
  row.names = FALSE,
  caption = "Staff names and countries (SQL + R)"
)
```

staff_id	staff_name	country
1	Mike Hillyer	Canada
2	Jon Stephens	Australia

Method 2 - Single SQL Query

```r
result_sql_b <- dbGetQuery(
  con,
```

```
  "SELECT
     s.staff_id,
     s.first_name || ' ' || s.last_name AS staff_name,
     co.country
   FROM staff AS s
   JOIN address AS a ON s.address_id = a.address_id
   JOIN city AS ci ON a.city_id = ci.city_id
   JOIN country AS co ON ci.country_id = co.country_id
   ORDER BY s.staff_id"
)

knitr::kable(
  result_sql_b,
  row.names = FALSE,
  caption = "Staff names and countries (SQL only)"
)
```

staff_id	staff_name	country
1	Mike Hillyer	Canada
2	Jon Stephens	Australia

Performance comparison

```
bench_b <- microbenchmark::microbenchmark(
  SQL_R = {
    staff_b <- dbGetQuery(
      con,
      "SELECT staff_id, first_name, last_name, address_id FROM staff"
    )

    address_b <- dbGetQuery(
      con,
      "SELECT address_id, city_id FROM address"
    )

    city_b <- dbGetQuery(
      con,
      "SELECT city_id, country_id FROM city"
    )

    country_b <- dbGetQuery(
      con,
```

```
      "SELECT country_id, country FROM country"
    )

    info_b <- merge(staff_b, address_b, by = "address_id")
    info_b <- merge(info_b, city_b, by = "city_id")
    info_b <- merge(info_b, country_b, by = "country_id")

    result_b <- data.frame(
      staff_id = info_b$staff_id,
      staff_name = paste(info_b$first_name, info_b$last_name),
      country = info_b$country
    )

    result_b[order(result_b$staff_id), ]
  },
  SQL_only = {
    dbGetQuery(
      con,
      "SELECT
         s.staff_id,
         s.first_name || ' ' || s.last_name AS staff_name,
         co.country
       FROM staff AS s
       JOIN address AS a ON s.address_id = a.address_id
       JOIN city AS ci ON a.city_id = ci.city_id
       JOIN country AS co ON ci.country_id = co.country_id
       ORDER BY s.staff_id"
    )
  },
  times = 100L
)

bench_summary_b <- summary(bench_b, unit = "us")

median_sql_r_b <- bench_summary_b$median[bench_summary_b$expr == "SQL_R"]
median_sql_only_b <- bench_summary_b$median[bench_summary_b$expr == "SQL_only"]

bench_summary_b
```

```
      expr    min      lq     mean  median      uq    max neval
1    SQL_R 1541.6 1630.30 1881.998 1706.55 1851.50 4365.0   100
2 SQL_only  161.1  187.05  243.421  224.45  251.05  722.5   100
```

The microbenchmark results show that the SQL-only approach is faster than the SQL + R approach. Over 100 evaluations, the median execution times were approximately 224.45 and 1706.55 microseconds, respectively. Thus, the SQL-only approach was about 7.60 times faster in this benchmark, likely because it performs the joins directly in the database, avoiding multiple data transfers and additional merging operations in R.

(c) Films rented for the highest dollar value

Method 1 - SQL + R

```
payment_data <- dbGetQuery(
  con,
  "SELECT rental_id, amount FROM payment"
)

rental_data <- dbGetQuery(
  con,
  "SELECT rental_id, inventory_id FROM rental"
)

inventory_data <- dbGetQuery(
  con,
  "SELECT inventory_id, film_id FROM inventory"
)

film_data <- dbGetQuery(
  con,
  "SELECT film_id, title FROM film"
)

film_payments <- merge(payment_data, rental_data, by = "rental_id")
film_payments <- merge(film_payments, inventory_data, by = "inventory_id")
film_payments <- merge(film_payments, film_data, by = "film_id")

max_amount <- max(film_payments$amount)

result_r_c <- unique(
  film_payments[film_payments$amount == max_amount, c("title", "amount")]
)

result_r_c <- result_r_c[order(result_r_c$title), ]

names(result_r_c) <- c("film_title", "payment_usd")
```

```r
rownames(result_r_c) <- NULL

knitr::kable(
  result_r_c,
  digits = 2,
  row.names = FALSE,
  caption = "Films rented for the highest dollar value (SQL + R)"
)
```

film_title	payment_usd
FLINTSTONES HAPPINESS	11.99
MIDSUMMER GROUNDHOG	11.99
MINE TITANS	11.99
SCORPION APOLLO	11.99
SHOW LORD	11.99
STING PERSONAL	11.99
TIES HUNGER	11.99
TRAP GUYS	11.99
VIRTUAL SPOILERS	11.99

Method 2 - Single SQL Query

```r
result_sql_c <- dbGetQuery(
  con,
  "SELECT DISTINCT
     f.title AS film_title,
     p.amount AS payment_usd
   FROM payment AS p
   JOIN rental AS r ON p.rental_id = r.rental_id
   JOIN inventory AS i ON r.inventory_id = i.inventory_id
   JOIN film AS f ON i.film_id = f.film_id
   WHERE p.amount = (SELECT MAX(amount) FROM payment)
   ORDER BY f.title"
)

knitr::kable(
  result_sql_c,
  digits = 2,
```

```
  row.names = FALSE,
  caption = "Films rented for the highest dollar value (SQL only)"
)
```

film_title	payment_usd
FLINTSTONES HAPPINESS	11.99
MIDSUMMER GROUNDHOG	11.99
MINE TITANS	11.99
SCORPION APOLLO	11.99
SHOW LORD	11.99
STING PERSONAL	11.99
TIES HUNGER	11.99
TRAP GUYS	11.99
VIRTUAL SPOILERS	11.99

Performance comparison

```
bench_c <- microbenchmark::microbenchmark(
  SQL_R = {
    payment_c <- dbGetQuery(
      con,
      "SELECT rental_id, amount FROM payment"
    )

    rental_c <- dbGetQuery(
      con,
      "SELECT rental_id, inventory_id FROM rental"
    )

    inventory_c <- dbGetQuery(
      con,
      "SELECT inventory_id, film_id FROM inventory"
    )

    film_c <- dbGetQuery(
      con,
      "SELECT film_id, title FROM film"
    )
```

```
    film_payments_c <- merge(payment_c, rental_c, by = "rental_id")
    film_payments_c <- merge(film_payments_c, inventory_c, by = "inventory_id")
    film_payments_c <- merge(film_payments_c, film_c, by = "film_id")

    max_amount_c <- max(film_payments_c$amount)

    result_c <- unique(
          film_payments_c[film_payments_c$amount  ==  max_amount_c,  c("title",
"amount")]
    )

    result_c <- result_c[order(result_c$title), ]

    names(result_c) <- c("film_title", "payment_usd")
    rownames(result_c) <- NULL

    result_c
  },
  SQL_only = {
    dbGetQuery(
      con,
      "SELECT DISTINCT
         f.title AS film_title,
         p.amount AS payment_usd
       FROM payment AS p
       JOIN rental AS r ON p.rental_id = r.rental_id
       JOIN inventory AS i ON r.inventory_id = i.inventory_id
       JOIN film AS f ON i.film_id = f.film_id
       WHERE p.amount = (SELECT MAX(amount) FROM payment)
       ORDER BY f.title"
    )
  },
  times = 100L
)

bench_summary_c <- summary(bench_c, unit = "us")

median_sql_r_c <- bench_summary_c$median[bench_summary_c$expr == "SQL_R"]
median_sql_only_c <- bench_summary_c$median[bench_summary_c$expr == "SQL_only"]

bench_summary_c
```

```
      expr     min       lq      mean   median      uq     max neval
1    SQL_R 32007.7 35263.90 37504.182 36621.05 37859.8 98779.5   100
2 SQL_only   818.2  1016.75  2287.842  1661.10  3398.7  4764.1   100
```

The microbenchmark results show that the SQL-only approach is faster than the SQL + R approach. Over 100 evaluations, the median execution times were approximately 1661.10 and 36621.05 microseconds, respectively. Thus, the SQL-only approach was about 22.05 times faster in this benchmark, likely because it performs the joins and filtering directly in the database, reducing data transfer and additional processing in R.

Problem 3 - Australian Records

```
australia <- read.csv("data/au-500.csv", stringsAsFactors = FALSE)

dim(australia)
```

```
[1] 500  11
```

```
names(australia)
```

```
 [1] "first_name"   "last_name"    "company_name" "address"      "city"
 [6] "state"        "post"         "phone1"       "phone2"       "email"
[11] "web"
```

```
head(australia)
```

```
  first_name last_name                   company_name               address
1   Rebbecca     Didio         Brandt, Jonathan F Esq         171 E 24th St
2     Stevie     Hallo     Landrum Temporary Services        22222 Acoma St
3     Mariko    Stayer            Inabinet, Macre Esq 534 Schoenborn St #51
4    Gerardo    Woodka       Morris Downing & Sherred     69206 Jackson Ave
5      Mayra      Bena             Buelt, David L Esq     808 Glen Cove Ave
6     Idella  Scotland Artesian Ice & Cold Storage Co      373 Lafayette St
         city state post       phone1       phone2                       email
1       Leith   TAS 7315 03-8174-9123 0458-665-290 rebbecca.didio@didio.com.au
2     Proston   QLD 4613 07-9997-3366 0497-622-620    stevie.hallo@hotmail.com
3       Hamel    WA 6215 08-5558-9019 0427-885-282   mariko_stayer@hotmail.com
4    Talmalmo   NSW 2640 02-6044-4682 0443-795-912  gerardo_woodka@hotmail.com
```

```
5   Lane Cove   NSW 1595 02-1455-6085 0453-666-885        mayra.bena@gmail.com
6 Cartmeticup    WA 6316 08-7868-1355 0451-966-921          idella@hotmail.com
                                          web
1       http://www.brandtjonathanfesq.com.au
2 http://www.landrumtemporaryservices.com.au
3         http://www.inabinetmacreesq.com.au
4     http://www.morrisdowningsherred.com.au
5           http://www.bueltdavidlesq.com.au
6 http://www.artesianicecoldstorageco.com.au
```

(a) Percentage of .com websites

```
is_com <- grepl("\\.com$", australia$web)

sum(is_com)
```

```
[1] 0
```

```
percent_com <- mean(is_com) * 100

percent_com
```

```
[1] 0
```

Among the 500 websites, none end in `.com`, giving a percentage of 0%. Websites ending in `.com.au` or other domain extensions are not counted as `.com` websites.

(b) Most common email domain

```
email_domains <- sub(".*@", "", australia$email)

domain_counts <- sort(table(email_domains), decreasing = TRUE)

head(domain_counts, 10)
```

```
email_domains
    hotmail.com       gmail.com       yahoo.com     agar.net.au    agney.net.au
            114             102              84               1               1
```

```
 ahlborn.com.au albrough.com.au   alerte.com.au   amedro.net.au  andrion.com.au
              1               1               1               1               1
```

```
most_common_domain <- domain_counts[domain_counts == max(domain_counts)]

most_common_domain
```

```
hotmail.com
        114
```

The most common email domain is `hotmail.com`, appearing 114 times among the 500 email addresses (22.8%). The next most common domains are `gmail.com` (102 occurrences) and `yahoo.com` (84 occurrences).

(c) Non-alphabetic characters in company names

Excluding commas and whitespace

```
nonalpha <- grepl("[^[:alpha:],[:space:]]", australia$company_name)

sum(nonalpha)
```

```
[1] 45
```

```
proportion_nonalpha <- mean(nonalpha)

proportion_nonalpha
```

```
[1] 0.09
```

Also excluding ampersands

```
nonalpha_no_amp <- grepl("[^[:alpha:],[:space:]&]", australia$company_name)

sum(nonalpha_no_amp)
```

```
[1] 4
```

```
proportion_no_amp <- mean(nonalpha_no_amp)

proportion_no_amp
```

```
[1] 0.008
```

Excluding commas and whitespace, 45 out of 500 company names contain at least one non-alphabetic character, giving a proportion of 0.09 (9%). When ampersands (&) are also excluded, only 4 company names meet this criterion, reducing the proportion to 0.008 (0.8%).

(d) Reformatting phone numbers

```
#' Convert Phone Numbers to Cell-Phone Format
#'
#' Reformats 10-digit phone numbers into the Australian
#' cell-phone format (XXXX-XXX-XXX).
#'
#' @param x A character vector of phone numbers.
#' @return A character vector of reformatted phone numbers.
#'
#' @examples
#' format_phone("03-8174-9123")

format_phone <- function(x) {
  digits <- gsub("[^0-9]", "", x)

  paste0(
    substr(digits, 1, 4), "-",
    substr(digits, 5, 7), "-",
    substr(digits, 8, 10)
  )
}

australia$phone1_cell <- format_phone(australia$phone1)
australia$phone2_cell <- format_phone(australia$phone2)

phone_preview <- head(australia[c("phone1_cell", "phone2_cell")], 10)

knitr::kable(
  phone_preview,
  row.names = FALSE,
```

```
  caption = "First 10 phone numbers in cell-phone format"
)
```

phone1_cell	phone2_cell
0381-749-123	0458-665-290
0799-973-366	0497-622-620
0855-589-019	0427-885-282
0260-444-682	0443-795-912
0214-556-085	0453-666-885
0878-681-355	0451-966-921
0865-228-931	0427-991-688
0252-269-402	0415-961-606
0731-849-989	0411-732-965
0868-904-661	0461-862-457

All phone numbers in both columns were reformatted into the cell-phone format `XXXX-XXX-XXX`. The original 10 digits were preserved, with only the positions of the hyphens changed. The first 10 reformatted phone numbers from each column are displayed above.

(e) Histogram of log apartment numbers

```
addresses <- trimws(australia$address)

apartment_text <- regmatches(
  addresses,
  regexpr("[0-9]+$", addresses)
)

apartment_numbers <- as.numeric(apartment_text)

apartment_numbers <- apartment_numbers[apartment_numbers > 0]

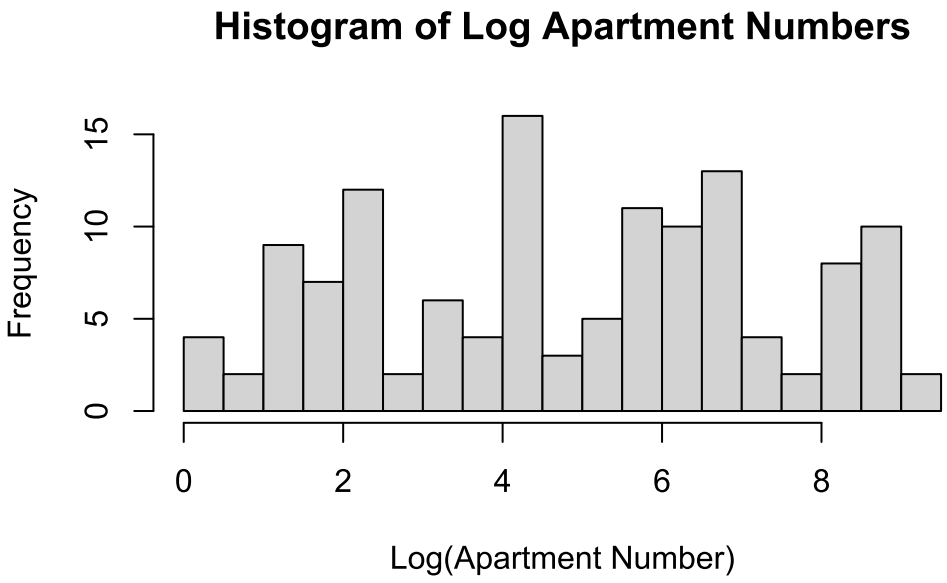
length(apartment_numbers)
```

```
[1] 130
```

```
summary(apartment_numbers)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   1.00    9.25  142.50 1154.57  943.25 8611.00
```

```r
hist(
  log(apartment_numbers),
  breaks = 20,
  main = "Histogram of Log Apartment Numbers",
  xlab = "Log(Apartment Number)",
  ylab = "Frequency"
)
```



A total of 130 valid positive apartment numbers were extracted from the addresses. These numbers range from 1 to 8,611, with a median of 142.5. The histogram shows the distribution of their natural logarithms. The log transformation compresses the larger values, making the distribution easier to visualize.

(f) Benford's law

```r
leading_digits <- substr(as.character(apartment_numbers), 1, 1)

digit_counts <- table(factor(leading_digits, levels = 1:9))
```

```r
observed_percent <- as.numeric(digit_counts) / sum(digit_counts) * 100

benford_probs <- log10(1 + 1 / (1:9))
benford_percent <- benford_probs * 100

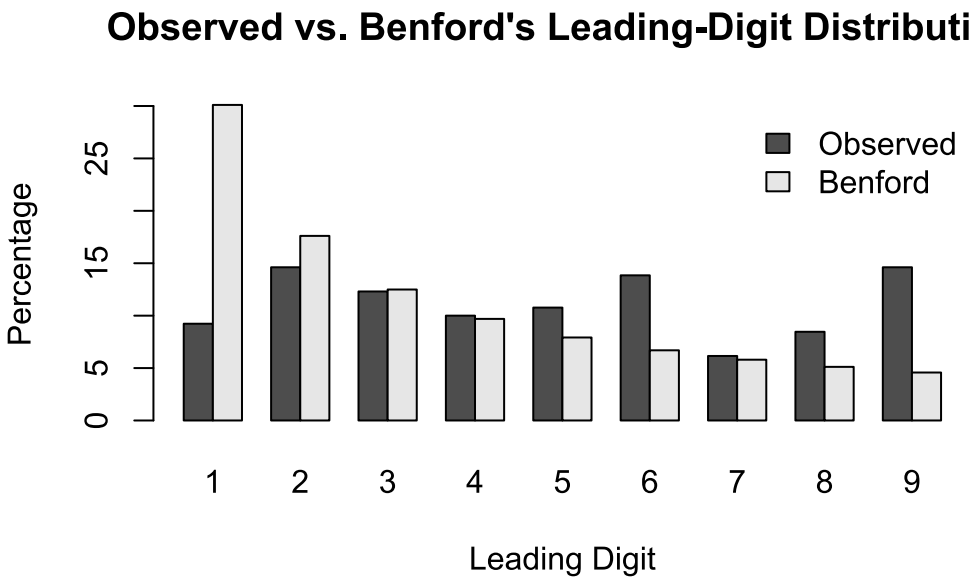
benford_table <- data.frame(
  Digit = 1:9,
  Count = as.integer(digit_counts),
  Observed_Percent = round(observed_percent, 2),
  Benford_Percent = round(benford_percent, 2)
)

knitr::kable(
  benford_table,
  row.names = FALSE,
  caption = "Observed leading-digit distribution vs. Benford's law"
)
```

Digit	Count	Observed_Percent	Benford_Percent
1	12	9.23	30.10
2	19	14.62	17.61
3	16	12.31	12.49
4	13	10.00	9.69
5	14	10.77	7.92
6	18	13.85	6.69
7	8	6.15	5.80
8	11	8.46	5.12
9	19	14.62	4.58

```r
barplot(
  rbind(
    Observed = observed_percent,
    Benford = benford_percent
  ),
  beside = TRUE,
  names.arg = 1:9,
  legend.text = TRUE,
  args.legend = list(x = "topright", bty = "n"),
  main = "Observed vs. Benford's Leading-Digit Distribution",
```

```
  xlab = "Leading Digit",
  ylab = "Percentage"
)
```




```
benford_test <- chisq.test(
  x = digit_counts,
  p = benford_probs
)

benford_test
```

```

    Chi-squared test for given probabilities

data:  digit_counts
X-squared = 62.268, df = 8, p-value = 1.67e-10
```

The observed leading-digit distribution differs substantially from Benford's law. For example, only 9.23% of the apartment numbers begin with 1, compared with the expected 30.10%, while 14.62% begin with 9, compared with the expected 4.58%.

The chi-square goodness-of-fit test gives a test statistic of 62.268 (df = 8, p-value = 1.67e-10). Since the p-value is much smaller than 0.05, we reject the null hypothesis that the leading digits follow Benford's law.

Therefore, these apartment numbers would not pass a Benford's-law-based check for real data. However, this does not necessarily prove that the data are fake, since real apartment numbers may also deviate from Benford's law due to numbering conventions and other constraints.